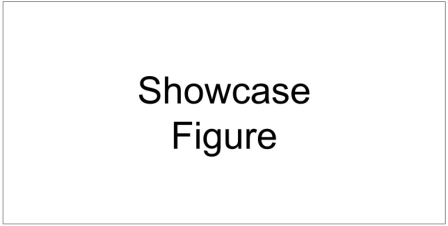
Understanding Trajectron++: A Per-Trajectory Performance Analysis

Simon Drauz and Zoe Matrullo

May 18, 2026



Outline

Outline

Trajectory Prediction for Autonomous Driving

- **Task:** forecast future (x, y) positions of agents (pedestrians, cyclists, vehicles) from observed past movements

[figure placeholder]

Trajectory Prediction for Autonomous Driving

- **Task:** forecast future (x, y) positions of agents (pedestrians, cyclists, vehicles) from observed past movements
- Challenging due to **complex agent interactions** and influence of **static context**

[figure placeholder]

Trajectory Prediction for Autonomous Driving

- **Task:** forecast future (x, y) positions of agents (pedestrians, cyclists, vehicles) from observed past movements
- Challenging due to **complex agent interactions** and influence of **static context**
- Essential for **safe path planning** and **collision avoidance** in autonomous vehicles

[figure placeholder]

Trajectron++

- State-of-the-art probabilistic trajectory prediction model [Salzmann et al., 2020]

Trajectron++

- State-of-the-art probabilistic trajectory prediction model [Salzmann et al., 2020]
- *Spatio-temporal graph encoder*

trajdata [NVLabs]

- Unified interface to many trajectory datasets (nuScenes, ETH/UCY, ...)

⇒ Our analysis pipeline applies to *any* trajdata-supported dataset

Trajectron++

- State-of-the-art probabilistic trajectory prediction model [Salzmann et al., 2020]
- *Spatio-temporal graph encoder*
  - LSTM per agent

trajdata [NVLabs]

- Unified interface to many trajectory datasets (nuScenes, ETH/UCY, ...)

$\Rightarrow$ Our analysis pipeline applies to *any* trajdata-supported dataset

Trajectron++

- State-of-the-art probabilistic trajectory prediction model [Salzmann et al., 2020]
- *Spatio-temporal graph encoder*
  - LSTM per agent
  - edges encode agent–agent interactions

trajdata [NVLabs]

- Unified interface to many trajectory datasets (nuScenes, ETH/UCY, . . .)

⇒ Our analysis pipeline applies to *any* trajdata-supported dataset

Trajectron++

- State-of-the-art probabilistic trajectory prediction model [Salzmann et al., 2020]
- *Spatio-temporal graph encoder*
  - LSTM per agent
  - edges encode agent–agent interactions
- *CVAE decoder*

trajdata [NVLabs]

- Unified interface to many trajectory datasets (nuScenes, ETH/UCY, ...)

$\Rightarrow$ Our analysis pipeline applies to *any* trajdata-supported dataset

Trajectron++

- State-of-the-art probabilistic trajectory prediction model [Salzmann et al., 2020]
- *Spatio-temporal graph encoder*
  - LSTM per agent
  - edges encode agent–agent interactions
- *CVAE decoder*
  - Outputs a distribution over future trajectories

trajdata [NVLabs]

- Unified interface to many trajectory datasets (nuScenes, ETH/UCY, ...)

$\Rightarrow$ Our analysis pipeline applies to *any* trajdata-supported dataset

Trajectron++

- State-of-the-art probabilistic trajectory prediction model [Salzmann et al., 2020]
- *Spatio-temporal graph encoder*
  - LSTM per agent
  - edges encode agent–agent interactions
- *CVAE decoder*
  - Outputs a distribution over future trajectories
- Handles *heterogeneous* agent types jointly (pedestrian, bicycle, car) and integrates HD map context

trajdata [NVLabs]

- Unified interface to many trajectory datasets (nuScenes, ETH/UCY, . . .)

$\Rightarrow$ Our analysis pipeline applies to *any* trajdata-supported dataset

nuScenes Dataset

- 1 000 urban driving scenes, Boston & Singapore

[figure placeholder]

nuScenes Dataset

- 1 000 urban driving scenes, Boston & Singapore
- Agent types: pedestrians, cyclists, vehicles

[figure placeholder]

nuScenes Dataset

- 1 000 urban driving scenes, Boston & Singapore
- Agent types: pedestrians, cyclists, vehicles
- Rich sensor suite: camera, LiDAR, radar, GPS, HD map

[figure placeholder]

nuScenes Dataset

- 1 000 urban driving scenes, Boston & Singapore
- Agent types: pedestrians, cyclists, vehicles
- Rich sensor suite: camera, LiDAR, radar, GPS, HD map
- Standard split: 700 / 150 / 150 (train / val / test)

[figure placeholder]

nuScenes Dataset

- 1 000 urban driving scenes, Boston & Singapore
- Agent types: pedestrians, cyclists, vehicles
- Rich sensor suite: camera, LiDAR, radar, GPS, HD map
- Standard split: 700 / 150 / 150 (train / val / test)
- **Scope:** we predict *pedestrian* trajectories only; all agent types serve as scene context

[figure placeholder]

nuScenes Dataset

- 1 000 urban driving scenes, Boston & Singapore
- Agent types: pedestrians, cyclists, vehicles
- Rich sensor suite: camera, LiDAR, radar, GPS, HD map
- Standard split: 700 / 150 / 150 (train / val / test)
- **Scope:** we predict *pedestrian* trajectories only; all agent types serve as scene context

[figure placeholder]

Prediction metrics:

`ml_ade`	most-likely ADE ← **used**
`ml_fde`	most-likely FDE
`min_ade_5`	best-of-5 ADE
`nll_mean`	negative log-likelihood

Challenges and Research Questions

- Some trajectories are inherently hard, others trivially easy—but *why?*

Challenges and Research Questions

- Some trajectories are inherently hard, others trivially easy—but *why?*
- A single average ADE/FDE gives no insight into reasons for failure and success

Challenges and Research Questions

- Some trajectories are inherently hard, others trivially easy—but *why?*
- A single average ADE/FDE gives no insight into reasons for failure and success
- No understanding of which trajectory or scene characteristics drive per-trajectory prediction error

Challenges and Research Questions

- Some trajectories are inherently hard, others trivially easy—but *why?*
- A single average ADE/FDE gives no insight into reasons for failure and success
- No understanding of which trajectory or scene characteristics drive per-trajectory prediction error

$\Rightarrow$ **Three research questions:**

Challenges and Research Questions

- Some trajectories are inherently hard, others trivially easy—but *why?*
- A single average ADE/FDE gives no insight into reasons for failure and success
- No understanding of which trajectory or scene characteristics drive per-trajectory prediction error

⇒ **Three research questions:**

1. Which trajectory and scene characteristics affect model performance?

Challenges and Research Questions

- Some trajectories are inherently hard, others trivially easy—but *why?*
- A single average ADE/FDE gives no insight into reasons for failure and success
- No understanding of which trajectory or scene characteristics drive per-trajectory prediction error

⇒ **Three research questions:**

1. Which trajectory and scene characteristics affect model performance?
2. What are the *success and failure modes* that explain model performance?

Challenges and Research Questions

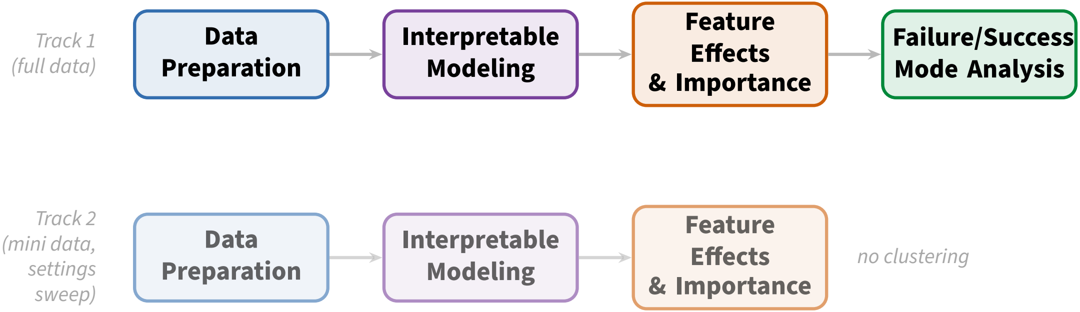
- Some trajectories are inherently hard, others trivially easy—but *why?*
- A single average ADE/FDE gives no insight into reasons for failure and success
- No understanding of which trajectory or scene characteristics drive per-trajectory prediction error

$\Rightarrow$ **Three research questions:**

1. Which trajectory and scene characteristics affect model performance?
2. What are the *success and failure modes* that explain model performance?
3. How do model settings (history window, prediction horizon, attention radius) affect performance?

Outline

Analysis Pipeline



Trajectron++ Modification

Standard Trajectron++

- Evaluation outputs *batch-averaged* ADE / FDE

Trajectron++ Modification

Standard Trajectron++

- Evaluation outputs *batch-averaged* ADE / FDE
- No way to link prediction error back to an individual trajectory

Trajectron++ Modification

Standard Trajectron++

- Evaluation outputs *batch-averaged* ADE / FDE
- No way to link prediction error back to an individual trajectory
- Cannot study what makes a trajectory hard or easy

Trajectron++ Modification

Standard Trajectron++

- Evaluation outputs *batch-averaged* ADE / FDE
- No way to link prediction error back to an individual trajectory
- Cannot study what makes a trajectory hard or easy

Our Modification

Trajectron++ Modification

Standard Trajectron++

- Evaluation outputs *batch-averaged* ADE / FDE
- No way to link prediction error back to an individual trajectory
- Cannot study what makes a trajectory hard or easy

Our Modification

- Recover *individual trajectory loss & characteristics*
  by modifying Trajectron++'s evaluation loop

Trajectron++ Modification

Standard Trajectron++

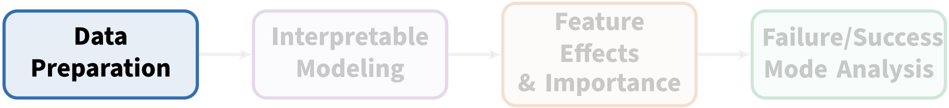
- Evaluation outputs *batch-averaged* ADE / FDE
- No way to link prediction error back to an individual trajectory
- Cannot study what makes a trajectory hard or easy

Our Modification

- Recover *individual trajectory loss & characteristics* by modifying Trajectron++'s evaluation loop
- `data_idx`: stable trajdata identifier, used to join evaluation output with computed trajectory & scene characteristics

Outline

Analysis Pipeline



Feature Engineering — Trajectory & Scene Characteristics

Trajectron++ outputs only (x, y) position sequences — all features below are computed by us from those trajectories.

Agent motion features

Feature Engineering — Trajectory & Scene Characteristics

Trajectron++ outputs only (x, y) position sequences — all features below are computed by us from those trajectories.

Agent motion features

- Speed: mean, max, std
- Acceleration: mean, max
- Jerk: mean, max
- Heading change per second
- Min. neighbour distance

Feature Engineering — Trajectory & Scene Characteristics

Trajectron++ outputs only (x, y) position sequences — all features below are computed by us from those trajectories.

Agent motion features

- Speed: mean, max, std
- Acceleration: mean, max
- Jerk: mean, max
- Heading change per second
- Min. neighbour distance

Scene context features

Feature Engineering — Trajectory & Scene Characteristics

Trajectron++ outputs only (x, y) position sequences — all features below are computed by us from those trajectories.

Agent motion features

- Speed: mean, max, std
- Acceleration: mean, max
- Jerk: mean, max
- Heading change per second
- Min. neighbour distance

Scene context features

- Number of agents in scene
- Number of vehicles
- Scene bounding box area
- Spatial agent density

Feature Engineering — Trajectory & Scene Characteristics

Trajectron++ outputs only (x, y) position sequences — all features below are computed by us from those trajectories.

Agent motion features

- Speed: mean, max, std
- Acceleration: mean, max
- Jerk: mean, max
- Heading change per second
- Min. neighbour distance

Scene context features

- Number of agents in scene
- Number of vehicles
- Scene bounding box area
- Spatial agent density

$\Rightarrow$ **18 candidate features** joined with Trajectron++ evaluation output via `data_idx`

Target Distribution & Log Transformation

- `ml_ade` is strongly *right-skewed* (skewness $= 2.59$)

[figure placeholder: before/after distribution]

Target Distribution & Log Transformation

- `ml_ade` is strongly *right-skewed* (skewness $= 2.59$)
- Log transformation (`log1p`) considered to stabilise variance — tested as a modeling choice in the next step

[figure placeholder: before/after distribution]

Feature Selection — Two Approaches

Approach 1: MI elbow + VIF

- Rank features by Mutual Information with target

Feature Selection — Two Approaches

Approach 1: MI elbow + VIF

- Rank features by Mutual Information with target
- Drop low-signal tail below the elbow: 18 $\rightarrow$ 10

Feature Selection — Two Approaches

Approach 1: MI elbow + VIF

- Rank features by Mutual Information with target
- Drop low-signal tail below the elbow: 18 $\rightarrow$ 10
- Apply VIF filter (threshold $=$ 5): 10 $\rightarrow$ **4 features**

Feature Selection — Two Approaches

Approach 1: MI elbow + VIF

- Rank features by Mutual Information with target
- Drop low-signal tail below the elbow: 18 $\rightarrow$ 10
- Apply VIF filter (threshold $= 5$): 10 $\rightarrow$ **4 features**

`std_speed`, `mean_acceleration`, `max_speed`, `heading_change_per_sec`

Motion features only ($p \ll 0.01$).

Feature Selection — Two Approaches

Approach 1: MI elbow + VIF

- Rank features by Mutual Information with target
- Drop low-signal tail below the elbow: 18 $\rightarrow$ 10
- Apply VIF filter (threshold $= 5$): 10 $\rightarrow$ **4 features**

`std_speed`, `mean_acceleration`, `max_speed`, `heading_change_per_sec`

Motion features only ($p \ll 0.01$).

Approach 2: VIF only

- Skip MI; apply VIF directly to all 18 candidates
- VIF filter (threshold $= 5$): 18 $\rightarrow$ **6 features**

Feature Selection — Two Approaches

Approach 1: MI elbow + VIF

- Rank features by Mutual Information with target
- Drop low-signal tail below the elbow: 18 $\rightarrow$ 10
- Apply VIF filter (threshold $= 5$): 10 $\rightarrow$ **4 features**

`std_speed`, `mean_acceleration`, `max_speed`, `heading_change_per_sec`

Motion features only ($p \ll 0.01$).

Approach 2: VIF only

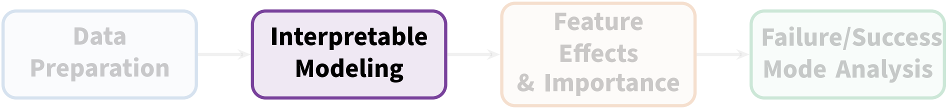
- Skip MI; apply VIF directly to all 18 candidates
- VIF filter (threshold $= 5$): 18 $\rightarrow$ **6 features**

`std_speed`, `mean_acceleration`, `min_neighbor_distance`, `heading_change_per_sec`, `scene_num_VEHICLE`, `scene_density_VEHICLE`

Retains scene-level features alongside motion features.

Outline

Analysis Pipeline



Why Interpretable Models?

- Per-trajectory `ml_ade` alone do not tell us *why* some trajectories are hard

Why Interpretable Models?

- Per-trajectory `ml_ade` alone do not tell us *why* some trajectories are hard
- Interpretable models let us directly read off feature effects: how does speed, heading change or scene density affect prediction error?

Why Interpretable Models?

- Per-trajectory `ml_ade` alone do not tell us *why* some trajectories are hard
- Interpretable models let us directly read off feature effects: how does speed, heading change or scene density affect prediction error?
- Two tested approaches:

Why Interpretable Models?

- Per-trajectory `ml_ade` alone do not tell us *why* some trajectories are hard
- Interpretable models let us directly read off feature effects: how does speed, heading change or scene density affect prediction error?
- Two tested approaches:
  - **GAM:** smooth effects directly readable from the model

Why Interpretable Models?

- Per-trajectory `ml_ade` alone do not tell us *why* some trajectories are hard
- Interpretable models let us directly read off feature effects: how does speed, heading change or scene density affect prediction error?
- Two tested approaches:
  - **GAM:** smooth effects directly readable from the model
  - **XGBoost:** post-hoc via SHAP values and partial dependence plots

Why Interpretable Models?

- Per-trajectory `ml_ade` alone do not tell us *why* some trajectories are hard
- Interpretable models let us directly read off feature effects: how does speed, heading change or scene density affect prediction error?
- Two tested approaches:
  - **GAM:** smooth effects directly readable from the model
  - **XGBoost:** post-hoc via SHAP values and partial dependence plots
- This turns ADE into **actionable understanding** of model behaviour

GAM and XGBoost

GAM (Generalized Additive Model)

Inherently interpretable

XGBoost

Black-box, interpreted post-hoc

GAM and XGBoost

GAM (Generalized Additive Model)

Inherently interpretable

- Models target as a sum of smooth functions: $f(x) = \sum_j s_j(x_j)$
- One spline per feature — no interaction terms

XGBoost

Black-box, interpreted post-hoc

GAM and XGBoost

GAM (Generalized Additive Model)

Inherently interpretable

- Models target as a sum of smooth functions: $f(x) = \sum_j s_j(x_j)$
- One spline per feature — no interaction terms
- p-values for significance testing

XGBoost

Black-box, interpreted post-hoc

GAM and XGBoost

GAM (Generalized Additive Model)

Inherently interpretable

- Models target as a sum of smooth functions: $f(x) = \sum_j s_j(x_j)$
- One spline per feature — no interaction terms
- p-values for significance testing
- Feature effects directly readable from the model

XGBoost

Black-box, interpreted post-hoc

GAM and XGBoost

GAM (Generalized Additive Model)

Inherently interpretable

- Models target as a sum of smooth functions: $f(x) = \sum_j s_j(x_j)$
- One spline per feature — no interaction terms
- p-values for significance testing
- Feature effects directly readable from the model

XGBoost

Black-box, interpreted post-hoc

- Gradient-boosted decision trees

GAM and XGBoost

GAM (Generalized Additive Model)

Inherently interpretable

- Models target as a sum of smooth functions: $f(x) = \sum_j s_j(x_j)$
- One spline per feature — no interaction terms
- p-values for significance testing
- Feature effects directly readable from the model

XGBoost

Black-box, interpreted post-hoc

- Gradient-boosted decision trees
- Flexibly captures non-linearities and feature interactions

GAM and XGBoost

GAM (Generalized Additive Model)

Inherently interpretable

- Models target as a sum of smooth functions: $f(x) = \sum_j s_j(x_j)$
- One spline per feature — no interaction terms
- p-values for significance testing
- Feature effects directly readable from the model

XGBoost

Black-box, interpreted post-hoc

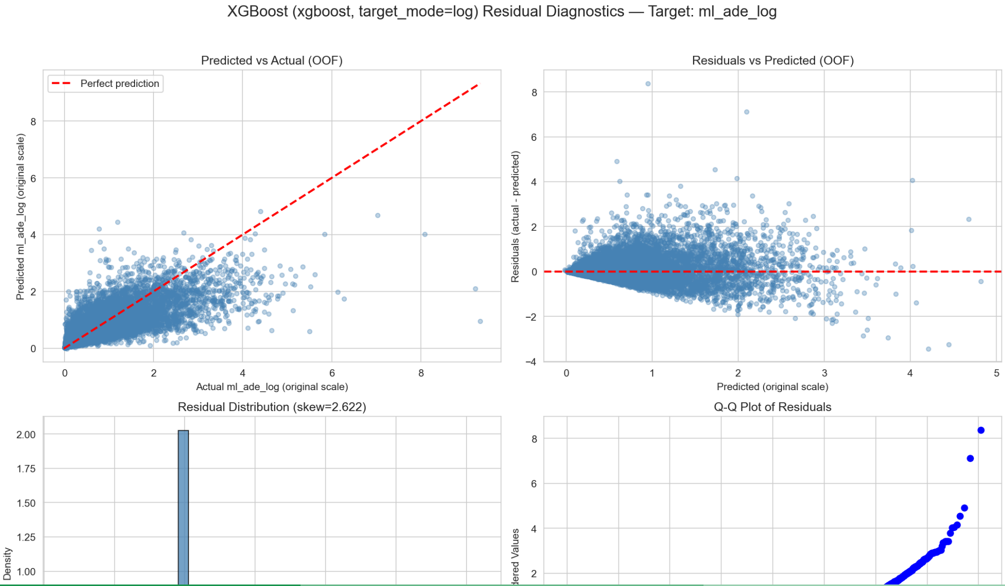
- Gradient-boosted decision trees
- Flexibly captures non-linearities and feature interactions
- Interpreted via **SHAP values** and **partial dependence plots**

Validating the Tuning Procedure — Nested Resampling

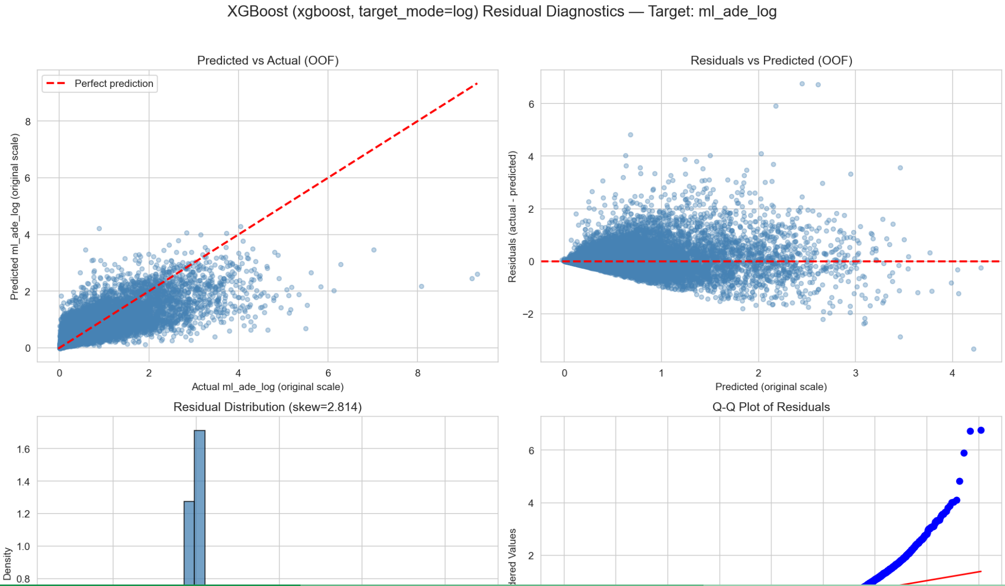
- **Problem:** hyperparameter tuning risks overfitting to a particular data split
- **Solution:** nested cross-validation
  - Outer folds: evaluate generalization of the full tuning procedure
  - Inner folds: tune hyperparameters on each outer-fold training set via cross-validation
- OOF evaluation: inspect out-of-fold predictions for instability and bias
- **Conclusion:** tuning generalises stably $\rightarrow$ proceed to full-data tuning + refit

Final models are refit on *all* data with optimal hyperparameters — no held-out test set, because the goal is understanding, not out-of-fold performance.

OOF Diagnostics — XGBoost (MI + VIF, 4 features)



OOF Diagnostics — XGBoost (VIF only, 6 features)



GAM Variant Selection

Three candidates:

- **LinearGAM** — fitted on raw `ml_ade`
- **LinearGAM (log)** — fitted on $\log(1 + \texttt{ml\_ade})$, evaluated on original scale
- **GammaGAM** — Gamma distribution with log link, suited for positive right-skewed targets

Selection criterion: lowest mean outer-fold RMSE (original scale).

Variant	RMSE	R^2	MAE
LinearGAM	**0.440**	**0.515**	0.268
LinearGAM (log)	0.444	0.506	**0.255**
GammaGAM	0.475	0.434	0.277

Outer-fold means, 3-fold nested CV.
LinearGAM selected.

Final Model Performance (OOF)

GAM (LinearGAM)

- Fitted on raw `ml_ade`
- Additive smooth terms, one per feature

Metric	OOF value
RMSE	0.440
R^2	0.515
MAE	0.268

XGBoost

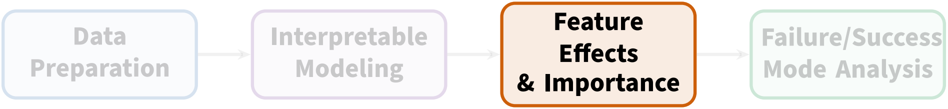
- Gradient-boosted trees
- Fitted on `log1p(ml_ade)`
- Post-hoc interpretation via SHAP

Metric	OOF value
RMSE	**0.387**
R^2	**0.624**
MAE	**0.213**

XGBoost outperforms GAM on all metrics. Both models are used as a cross-check: the feature-effect narratives should agree across approaches.

Outline

Analysis Pipeline



Global Feature Importance

XGBoost — mean $|\mathrm{SHAP}|$

Feature	**Mean** $|\mathrm{SHAP}|$
`std_speed`	**0.129**
`heading_change_per_sec`	0.070
`min_neighbor_distance`	0.027
`mean_acceleration`	0.027
`scene_density_VEHICLE`	0.020
`scene_num_VEHICLE`	0.016

GAM — smooth-term significance

Feature	$p < 0.05$**?**
`heading_change_per_sec`	✓
`mean_acceleration`	✓
`min_neighbor_distance`	✓
`std_speed`	✓
`scene_num_VEHICLE`	✓
`scene_density_VEHICLE`	×

Both models agree: speed variation (`std_speed`) and turning (`heading_change_per_sec`) are the dominant drivers of prediction error. Scene density is consistently least important.

Feature Effects — What Drives Prediction Error?

Motion features

- `std_speed`: strong positive effect — erratic speed changes make trajectories harder to forecast

Feature Effects — What Drives Prediction Error?

Motion features

- `std_speed`: strong positive effect — erratic speed changes make trajectories harder to forecast
- `heading_change_per_sec`: positive effect — more turning $\rightarrow$ higher ADE

Feature Effects — What Drives Prediction Error?

Motion features

- `std_speed`: strong positive effect — erratic speed changes make trajectories harder to forecast
- `heading_change_per_sec`: positive effect — more turning $\rightarrow$ higher ADE
- `mean_acceleration`: positive effect — higher acceleration raises predicted error

Scene & proximity features

- `min_neighbor_distance`: mild positive effect — isolated agents are marginally harder to predict
- `scene_num_VEHICLE`: small positive effect; crowded vehicle context increases error slightly
- `scene_density_VEHICLE`: not significant in GAM; weakest SHAP contribution in XGBoost

Feature Effects — What Drives Prediction Error?

LMU LUDWIG-MAXIMILIANS-UNIVERSITÄT MÜNCHEN

Motion features

- `std_speed`: strong positive effect — erratic speed changes make trajectories harder to forecast
- `heading_change_per_sec`: positive effect — more turning $\rightarrow$ higher ADE
- `mean_acceleration`: positive effect — higher acceleration raises predicted error

Scene & proximity features

- `min_neighbor_distance`: mild positive effect — isolated agents are marginally harder to predict
- `scene_num_VEHICLE`: small positive effect; crowded vehicle context increases error slightly
- `scene_density_VEHICLE`: not significant in GAM; weakest SHAP contribution in XGBoost

$\Rightarrow$ Trajectron++ struggles most with **fast, erratic, turning pedestrians**; scene context is a secondary factor.

Feature Effects — Partial Dependence

GAM — smooth effects

- Marginal effect of each feature on predicted `ml_ade`, holding others fixed

Feature Effects — Partial Dependence

GAM — smooth effects

- Marginal effect of each feature on predicted `ml_ade`, holding others fixed
- `std_speed`: monotonically increasing — more speed variation $\rightarrow$ higher error

XGBoost — SHAP dependence

- Global picture consistent with GAM: `std_speed` contributes most strongly
- `min_neighbor_distance` shows a non-monotonic pattern — very close *and* very distant neighbors raise error

Feature Effects — Partial Dependence

GAM — smooth effects

- Marginal effect of each feature on predicted `ml_ade`, holding others fixed
- `std_speed`: monotonically increasing — more speed variation $\rightarrow$ higher error
- `heading_change_per_sec`: positive, non-linear — effect levels off at high turning rates

XGBoost — SHAP dependence

- Global picture consistent with GAM: `std_speed` contributes most strongly
- `min_neighbor_distance` shows a non-monotonic pattern — very close *and* very distant neighbors raise error
- Both models: scene features show flat or near-zero marginal effects compared to motion features

Feature Effects — Partial Dependence

GAM — smooth effects

- Marginal effect of each feature on predicted `ml_ade`, holding others fixed
- `std_speed`: monotonically increasing — more speed variation $\rightarrow$ higher error
- `heading_change_per_sec`: positive, non-linear — effect levels off at high turning rates
- Confidence bands narrow in high-density regions

XGBoost — SHAP dependence

- Global picture consistent with GAM: `std_speed` contributes most strongly
- `min_neighbor_distance` shows a non-monotonic pattern — very close *and* very distant neighbors raise error
- Both models: scene features show flat or near-zero marginal effects compared to motion features

Outline

Analysis Pipeline

Data Preparation → Interpretable Modeling → Feature Effects & Importance → **Failure/Success Mode Analysis**

Performance Regime Methodology

1. Divide trajectories into difficulty groups based on `ml_ade` (easy / medium / hard)
2. Run clustering on trajectory + scene characteristics per group
   - Option A: original feature space
   - Option B: UMAP-reduced space (dimensions chosen via trustworthiness score + elbow rule)
3. Assess clustering quality with DBCV score
4. Export identified clusters (performance regimes)

Clustering Results

Performance group sizes (VIF-only run)

Group	Boundary	N	%
Easy	< 0.39	15 348	59%
Medium	0.39–1.20	8 094	31%
Hard	> 1.20	2 680	10%

Clustering Results

Performance group sizes (VIF-only run)

Group	Boundary	N	%
Easy	< 0.39	15 348	59%
Medium	0.39–1.20	8 094	31%
Hard	> 1.20	2 680	10%

- Clustering algorithm: **OPTICS** on raw feature-effect space (no UMAP needed for 6-dimensional space)
- Quality assessed via **DBCV** score

Clustering Results

LMU LUDWIG-MAXIMILIANS-UNIVERSITÄT MÜNCHEN

Performance group sizes (VIF-only run)

Group	**Boundary**	**N**	**%**
Easy	< 0.39	15 348	59%
Medium	0.39–1.20	8 094	31%
Hard	> 1.20	2 680	10%

- Clustering algorithm: **OPTICS** on raw feature-effect space (no UMAP needed for 6-dimensional space)
- Quality assessed via **DBCV** score

Cluster quality by group

Group	**DBCV**	**Noise %**
Easy	**0.411**	30%
Medium	0.397	40%
Hard	0.091	75%

Easy group has the clearest substructure.
Hard is mostly a continuous high-error regime.

Regime Inspection — What Drives Error in Each Regime?

Easy regime

- Low `std_speed` (-2.0σ global) and low heading change $\rightarrow$ slow, predictable trajectories

Regime Inspection — What Drives Error in Each Regime?

Easy regime

- Low `std_speed` (-2.0σ global) and low heading change $\rightarrow$ slow, predictable trajectories
- Two sub-clusters: *slow/stationary* (median ADE 0.02) vs. *mild motion* (median ADE 0.22)

Hard regime

- 99.5% of hard trajectories have *positive* total feature effect
- Dominated by high `std_speed` ($+2.0\sigma$ global), turning, and acceleration

Regime Inspection — What Drives Error in Each Regime?

LMU LUDWIG-MAXIMILIANS-UNIVERSITÄT MÜNCHEN

Easy regime

- Low `std_speed` (-2.0σ global) and low heading change $\rightarrow$ slow, predictable trajectories
- Two sub-clusters: *slow/stationary* (median ADE 0.02) vs. *mild motion* (median ADE 0.22)
- Both drive negative total feature effects

Hard regime

- 99.5% of hard trajectories have *positive* total feature effect
- Dominated by high `std_speed` ($+2.0\sigma$ global), turning, and acceleration
- No sharp sub-clusters — hard is a **continuous high-variation regime**; a small extreme outlier group has `std_speed` $\approx +4.8\sigma$

Takeaway: Trajectron++ struggles with erratic, fast-turning pedestrians. Slow or near-stationary pedestrians are easy regardless of scene complexity.

Outline

Settings Sweep Setup

- Dataset: nuScenes mini (full dataset too expensive for repeated runs)
- Cartesian sweep over:
  - `history_sec`
  - `prediction_sec`
  - `attention_radius_m`
- Pooled model: model setting variables + trajectory/scene features as control variables
- Feature normalization: setting-agnostic versions to avoid mechanical confounding (e.g. `mean_path_length_per_second` instead of `path_length`)

Effect of Model Settings

XGBoost — mean $|\text{SHAP}|$ (pooled model)

Feature	Mean $|\text{SHAP}|$
`prediction_sec`	**0.178**
`attention_radius_m`	0.098
`std_speed`	0.093
`max_speed`	0.070
`history_sec`	0.022

All 8 features significant in GAM ($p \ll 0.01$).

Key findings

- `prediction_sec`: largest effect — longer horizon means harder forecasting (ADE grows with time)

Effect of Model Settings

XGBoost — mean $|\text{SHAP}|$ (pooled model)

Feature	Mean $|\text{SHAP}|$
prediction_sec	**0.178**
attention_radius_m	0.098
std_speed	0.093
max_speed	0.070
history_sec	0.022

All 8 features significant in GAM ($p \ll 0.01$).

Key findings

- prediction_sec: largest effect — longer horizon means harder forecasting (ADE grows with time)
- attention_radius_m: second — larger radius provides more context but also adds noise; non-linear optimal range

Effect of Model Settings

XGBoost — mean |SHAP| (pooled model)

Feature	**Mean** $\lvert \mathrm{SHAP} \rvert$
`prediction_sec`	**0.178**
`attention_radius_m`	0.098
`std_speed`	0.093
`max_speed`	0.070
`history_sec`	0.022

All 8 features significant in GAM ($p \ll 0.01$).

Key findings

- `prediction_sec`: largest effect — longer horizon means harder forecasting (ADE grows with time)
- `attention_radius_m`: second — larger radius provides more context but also adds noise; non-linear optimal range
- `history_sec`: smallest setting effect — model is robust to history window length in this range

Effect of Model Settings

XGBoost — mean $|\mathrm{SHAP}|$ (pooled model)

Feature	Mean $|\mathrm{SHAP}|$
`prediction_sec`	**0.178**
`attention_radius_m`	0.098
`std_speed`	0.093
`max_speed`	0.070
`history_sec`	0.022

All 8 features significant in GAM ($p \ll 0.01$).

Key findings

- `prediction_sec`: largest effect — longer horizon means harder forecasting (ADE grows with time)
- `attention_radius_m`: second — larger radius provides more context but also adds noise; non-linear optimal range
- `history_sec`: smallest setting effect — model is robust to history window length in this range
- Trajectory features (`std_speed`, `max_speed`) retain strong explanatory power even after controlling for settings

Outline

Summary of Findings

1. **RQ1 — Which characteristics drive prediction error?**
   - `std_speed` (speed variation) and `heading_change_per_sec` (turning) are the dominant drivers in both GAM and XGBoost
   - Scene-level features (`scene_density_VEHICLE`) are consistently least important; Trajectron++ handles scene context better than individual motion dynamics

Summary of Findings

1. **RQ1 — Which characteristics drive prediction error?**
   - `std_speed` (speed variation) and `heading_change_per_sec` (turning) are the dominant drivers in both GAM and XGBoost
   - Scene-level features (`scene_density_VEHICLE`) are consistently least important; Trajectron++ handles scene context better than individual motion dynamics
2. **RQ2 — What are the success and failure modes?**
   - Easy: slow or near-stationary pedestrians with low speed variation — Trajectron++ excels here
   - Hard: continuous high-variation regime driven by erratic, fast-turning pedestrians; no sharp sub-clusters

Summary of Findings

1. **RQ1 — Which characteristics drive prediction error?**
   - `std_speed` (speed variation) and `heading_change_per_sec` (turning) are the dominant drivers in both GAM and XGBoost
   - Scene-level features (`scene_density_VEHICLE`) are consistently least important; Trajectron++ handles scene context better than individual motion dynamics
2. **RQ2 — What are the success and failure modes?**
   - Easy: slow or near-stationary pedestrians with low speed variation — Trajectron++ excels here
   - Hard: continuous high-variation regime driven by erratic, fast-turning pedestrians; no sharp sub-clusters
3. **RQ3 — How do model settings affect performance?**
   - Prediction horizon (`prediction_sec`) has the largest effect — longer horizon raises error strongly
   - Attention radius (`attention_radius_m`) is second; history window (`history_sec`) has minimal impact

Limitations & Future Work

Limitations

- Analysis restricted to *pedestrian* trajectories on nuScenes — generalisability to other agent types or datasets is not guaranteed

Limitations & Future Work

Limitations

- Analysis restricted to *pedestrian* trajectories on nuScenes — generalisability to other agent types or datasets is not guaranteed
- Meta-models explain Trajectron++ predictions, not ground-truth difficulty — effects are model-specific

Limitations & Future Work

Limitations

- Analysis restricted to *pedestrian* trajectories on nuScenes — generalisability to other agent types or datasets is not guaranteed
- Meta-models explain Trajectron++ predictions, not ground-truth difficulty — effects are model-specific
- Hard-regime clustering is weak (75% noise); discrete failure-mode identification remains an open challenge

Future Work

Limitations & Future Work

Limitations

- Analysis restricted to *pedestrian* trajectories on nuScenes — generalisability to other agent types or datasets is not guaranteed
- Meta-models explain Trajectron++ predictions, not ground-truth difficulty — effects are model-specific
- Hard-regime clustering is weak (75% noise); discrete failure-mode identification remains an open challenge
- Settings sweep uses nuScenes *mini* — results may not transfer to full-scale training

Future Work

- Extend pipeline to vehicle and cyclist trajectories
- Apply to other trajectory prediction models and datasets via the trajdata interface
- Explore higher-capacity meta-models to better isolate failure modes in the hard regime